

Task3: Randomness-Related Vulnerability Identification

March 22, 2026

3.1 Static Analysis Results

Static analysis of the decompiled APK identified two locations related to random value generation. The first is `MainActivity.randomNumberGenerator()`, which uses `java.util.Random` to generate an integer between 0 and 99. The second is `Login.generateSessionToken()`, which also uses `java.util.Random`, but generates a 16-character alphanumeric string.

3.2 Security Role Analysis

The value generated in `MainActivity.randomNumberGenerator()` appears to be a general-purpose random integer. It is not clearly used in login, authorization, or session control. Therefore, it is not treated as a security-sensitive value. In contrast, the value generated in `Login.generateSessionToken()` is stored in `KEY_SESSION_TOKEN` through `createSession()` after successful login. This shows that it is used as a session token. Since a session token is a session credential, it should be treated as security-sensitive.

The value generated in `MainActivity.randomNumberGenerator()` appears to be a general-purpose random integer. It is not clearly involved in login, authorization, or session control, so it is not treated as security-sensitive. In contrast, the value generated in `Login.generateSessionToken()` is stored in `KEY_SESSION_TOKEN` through `createSession()` after a successful login. This indicates that it is used as a session token and should therefore be treated as security-sensitive.

Location	Generator API	Generated Value	Security Role
<code>MainActivity.randomNumberGenerator()</code>	<code>java.util.Random</code>	Integer between 0 and 99	General random value / UI use
<code>Login.generateSessionToken()</code>	<code>java.util.Random</code>	16-character alphanumeric string	Session token / session credential

Table 1. Randomness-Related Locations

3.3 Core Vulnerability Determination

Not every use of `Random` is a vulnerability. The key question is whether the generated value is used in a security-sensitive context (OWASP Foundation, n.d.). In this APK, the random number in `MainActivity` has no clear security role. However, `Login.generateSessionToken()` generates a session token that is stored for session use. Because `java.util.Random` is not intended for security-sensitive use, generating a session token with it reduces token unpredictability and weakens session security (OWASP Foundation, n.d.). Therefore, `Login.generateSessionToken()` is selected as the core vulnerability because it uses `java.util.Random` to generate a security-sensitive session token.

3.4 Fix and Mitigation

A suitable fix is to replace `java.util.Random` with `java.security.SecureRandom` when generating the session token. `SecureRandom` is more appropriate for security-sensitive values because it provides stronger unpredictability. The token length and entropy should also be increased, for example by generating more random bytes and encoding them as Base64 or hexadecimal. This would make the session token harder to predict and therefore improve session security.

Reference

[1] OWASP MAS. (n.d.). *MASTG-TEST-0204: Insecure random API usage*. OWASP Mobile Application Security Testing Guide.

<https://mas.owasp.org/MASTG/tests/android/MASVS-CRYPTO/MASTG-TEST-0204/>